

COS214 PRACTICAL 3

u25047762_u25243269_u25370325

TASK1: EVENT CONCEPT AND DESCRIPTION

- 1.1) Circuit Control is a one-day motorsport race weekend held at a permanent circuit, coordinated by a central race control system. Race control issues flags and instructions a red flag to stop a session, a yellow flag for local caution, a weather alert as conditions change, a capacity alert as grandstands fill, a medical incident call, or a resume once it's safe to continue and every operational unit on the circuit must react, each in its own way.

Five concrete kinds of event unit:

- MarshalPost - trackside flag point; tracks its own flag colour and an incident counter, requests reinforcement after repeated red flags, auto escalates to red under severe weather.
- TimingPoint – lap timing loop; freezes/resumes timing on red flag, records laps and tracks the best lap.
- PitGarage - pit lane garage bay; holds a bounded FIFO queue of cars, closes/opens its lane, applies an independent weather speed limit.
- RunOffArea - gravel run-off zone; closes while marshals work on track during a red flag.
- MedicalStation - medical response point; never fully closes, escalates between standby and deployed instead.

Three kinds of grouping/area in the Composite tree:

- Circuit – top level venue composite containing the racing-side zones.
- Track Zone / Pit Zone - functional sub-zones inside Circuit (marshal posts and timing points vs. pit garages).
- Spectator Area and Race Administration – purpose-built areas for grandstands and medical staff.

- 1.2) GoF participant mapping:

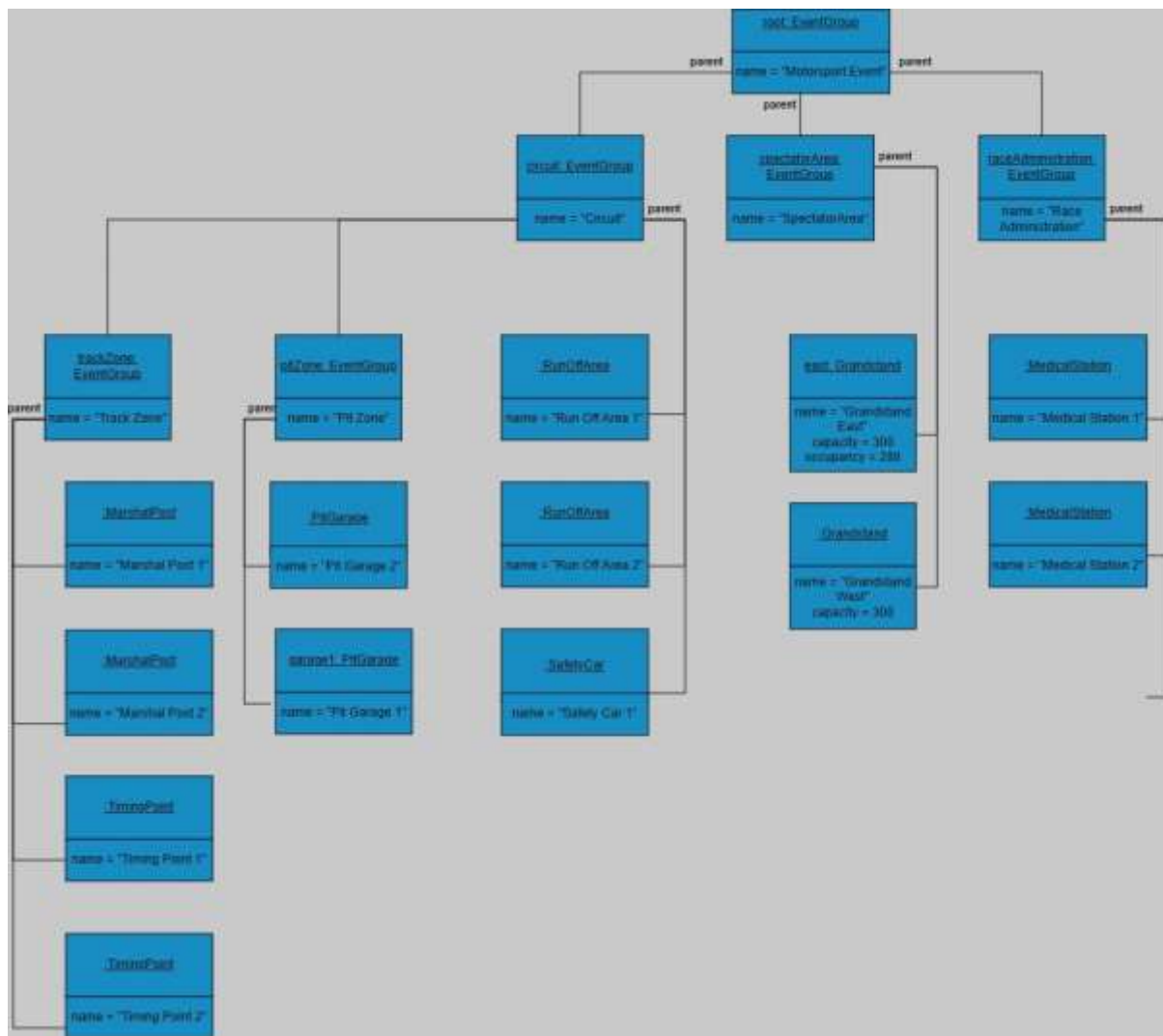
Composite:

ROLE	CLASS
Component	EventComponent
Composite	EventGroup
Leaf	MarshalPost, TimingPoint, PitGarage, RunOffArea, MedicalStation, Grandstand, SafetyCar

- b) Why Observer is required rather than direct hard coded calls: Race control shouldn't need to know every marshal post, timing loop, and pit garage individually, and adding a new unit type shouldn't require changing race control's code. Observer lets a single notice reach whatever is currently registered, including units added, removed, or reorganised at runtime.
- c) Who owns event components: Ownership follows the Composite tree exactly an EventGroup owns every child in its m_children vector and deletes each one in its destructor. The root is owned by the client (main()). EventControl is explicitly not an owner; it holds only a non-owning pointer to the root.
- d) Who owns (or doesn't own) observer references: Subject stores non owning Observer* pointers and never deletes an observer. The parent that owns a component is the same parent that registers it as an observer of itself (via auto registration in add()), so ownership and observer-registration end together when that parent is destroyed no dangling pointer is left behind. During a deliberate transfer between parents, remove() detaches the old registration before add() creates the new one.
- e) Why being both Subject and Observer isn't a misuse: EventGroup's Observer role serves its parent; its Subject role serves its children. Two separate one to many relationships, over two separate sets of collaborators, triggered at two separate moments not the same collaboration wearing two hats.

TASK2: Composite event structure

2.3) Object Diagram:



TASK3: Observer notification system

3.1) Registration policy

Subject::attach() is idempotent: it checks std::find() first and silently no ops if the pointer is already registered, so double attaching the same observer can't create duplicate notifications. detach() uses the erase-remove function and is also a safe no-op if the pointer isn't currently registered this single implementation covers both "detach twice" and "detach something never attached," with no special casing needed.

3.2) Our destruction policy is lifetime ordering by construction, not "observers detach themselves" and not "subjects clean up stale pointers." The only way anything is ever destroyed is delete root, which cascades through EventGroup's destructor and deletes the entire owned subtree in one recursive sweep. Since a Subject and every Observer

registered with it are always destroyed together in that same cascade, no Subject ever survives long enough to call notify() on a deleted object.

3.4) Cascading through 3+ levels

EventGroup::update() is the Observer-then-Subject hop: on receiving a notice from its parent, it calls its own notify(), forwarding to its own children. Since EventGroup implements both Observer and Subject, this same method runs at every level of the tree. Verified in Main.cpp: control, circuit trackZone, {marshalPost1, marshalPost2, timingPoint1, timingPoint2} is a real four-level runtime cascade from a single issueNotice() call.

3.5) Push vs Pull:

We chose push. Subject::notify () passes a complete, immutable EventNotice (type, sector, message, severity) by const reference straight into update (). No Subject caches a "current notice," and no Observer queries back into the Subject for state everything an observer needs to react arrives with the call, which is what lets a MarshalPost decide its own reaction (e.g. only escalating on WEATHER_ALERT if severity >= 4) without a second round-trip.

TASK4: Event rules and dynamic behaviour

4.1) On WEATHER_ALERT: MarshalPost auto escalates to red only if severity >= 4 (else just monitors); PitGarage unconditionally imposes a pit-lane speed limit; Grandstand and MedicalStation don't react to weather at all, no type checking anywhere, purely each class's own update() body. On RED_FLAG: TimingPoint freezes lap timing; MarshalPost escalates and increments an incident counter; MedicalStation stays operational per its close() override which never fully closes ("medical never fully closes").

4.2) Runtime reorganisation - Pit Garage 1 moves from Pit Zone to Race Administration: pitZone remove(garage1) releases ownership and detaches the observer registration; raceAdministration add(garage1) takes ownership back and re-attaches it, so it's reachable through Race Administration's cascade afterward.

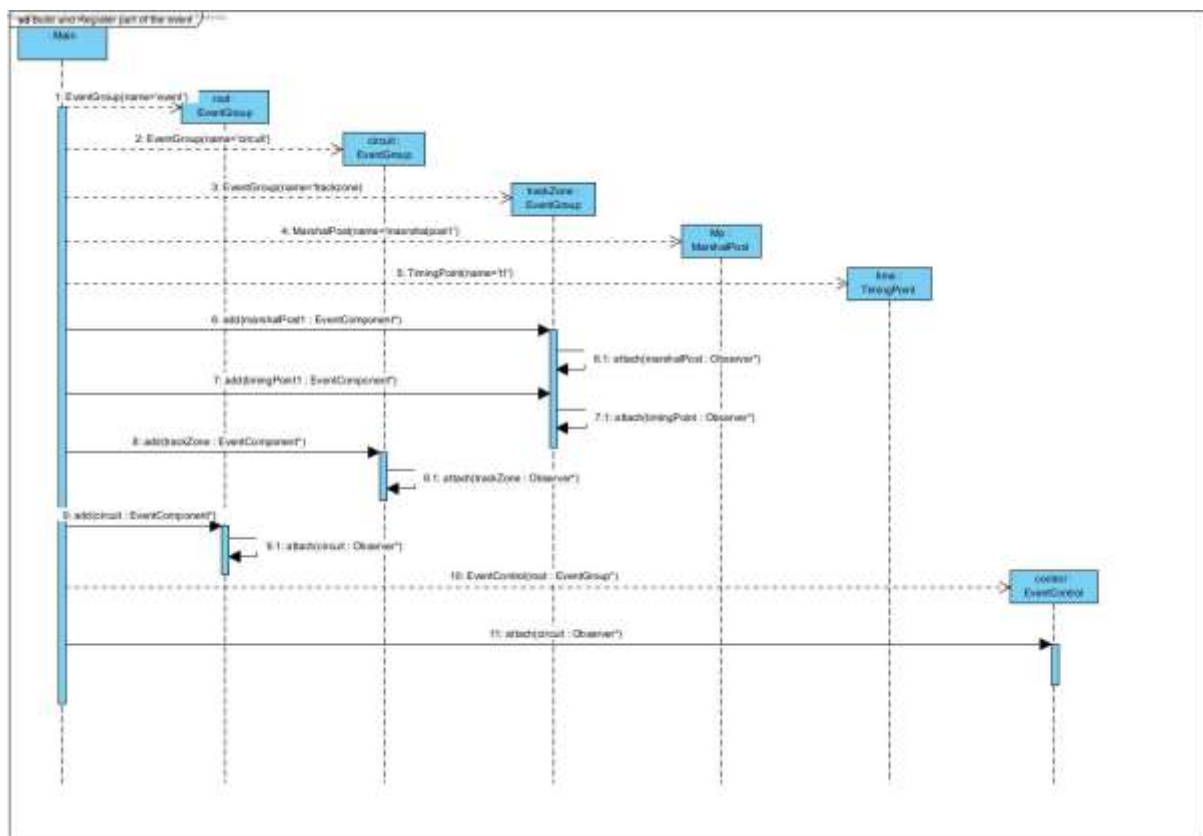
4.3) Condition-based decisions: Grandstand::update (CAPACITY_ALERT) branches on [occupancy/capacity >= 0.9]/[else]; EventGroup::checkCapacityThreshold () branches on [occupancy >= capacity]/[else]; MarshalPost::update (WEATHER_ALERT) branches on [severity >= 4]/[else].

4.4) Three original features, no god object

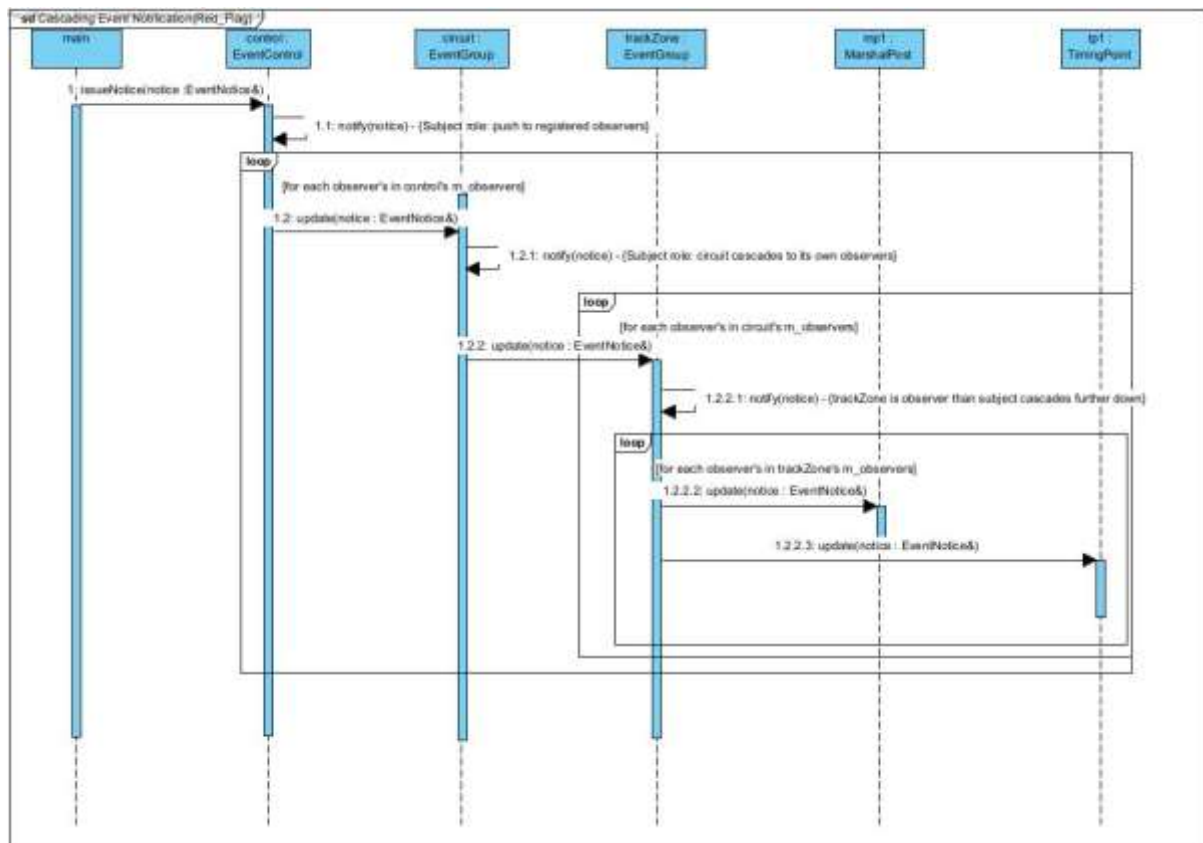
1. SafetyCar a seventh concrete leaf, standalone (doesn't touch any other class), adds a new interaction without adding coupling.
2. MarshalPost incident escalation a self-contained counter and threshold (REINFORCEMENT_THRESHOLD) entirely inside MarshalPost; no other class needs to know about it.
3. PitGarage bounded release queue capped queue logic fully contained inside PitGarage, doesn't require EventGroup or Subject to change.

TASK5: Sequence diagrams

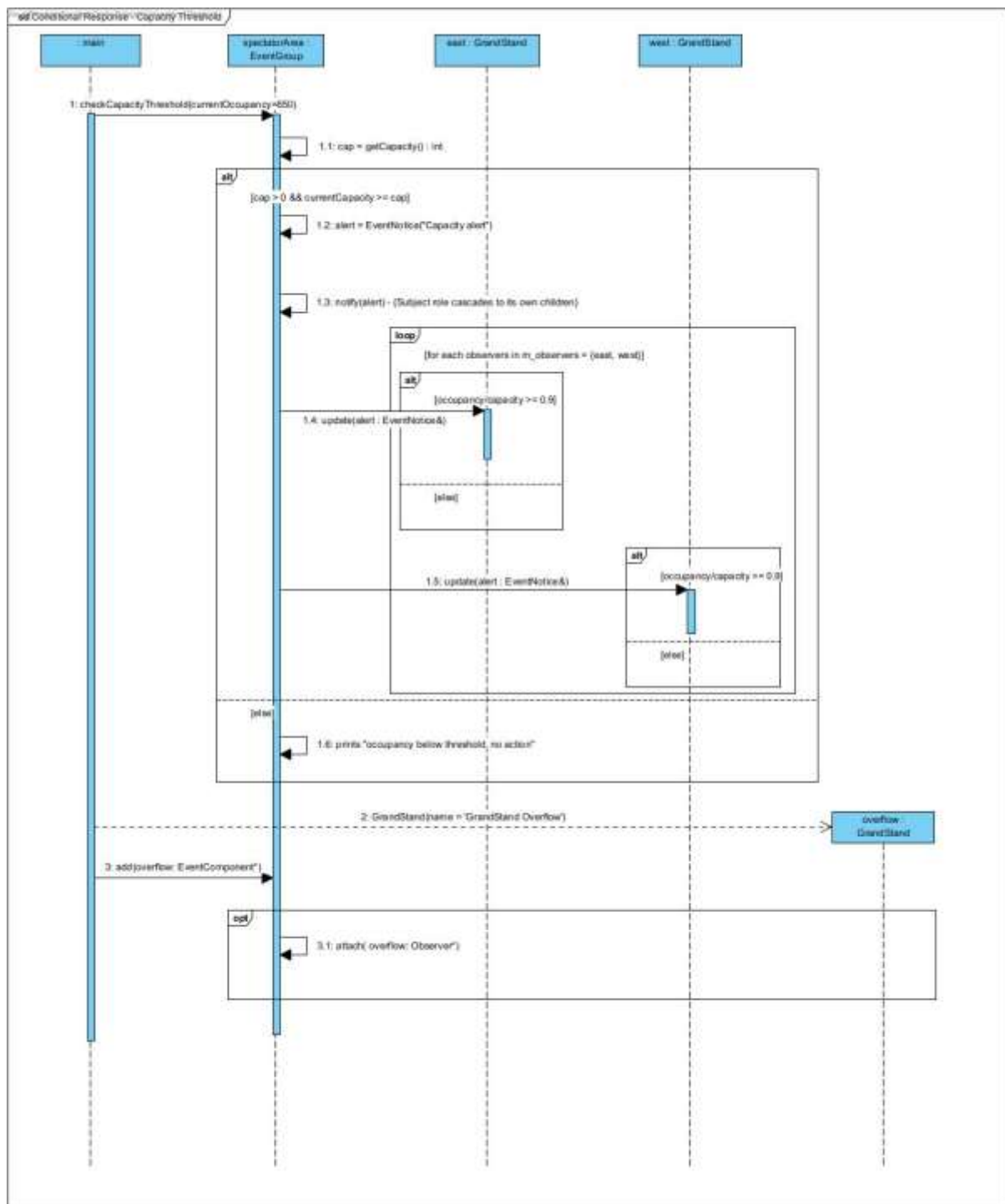
5.1)



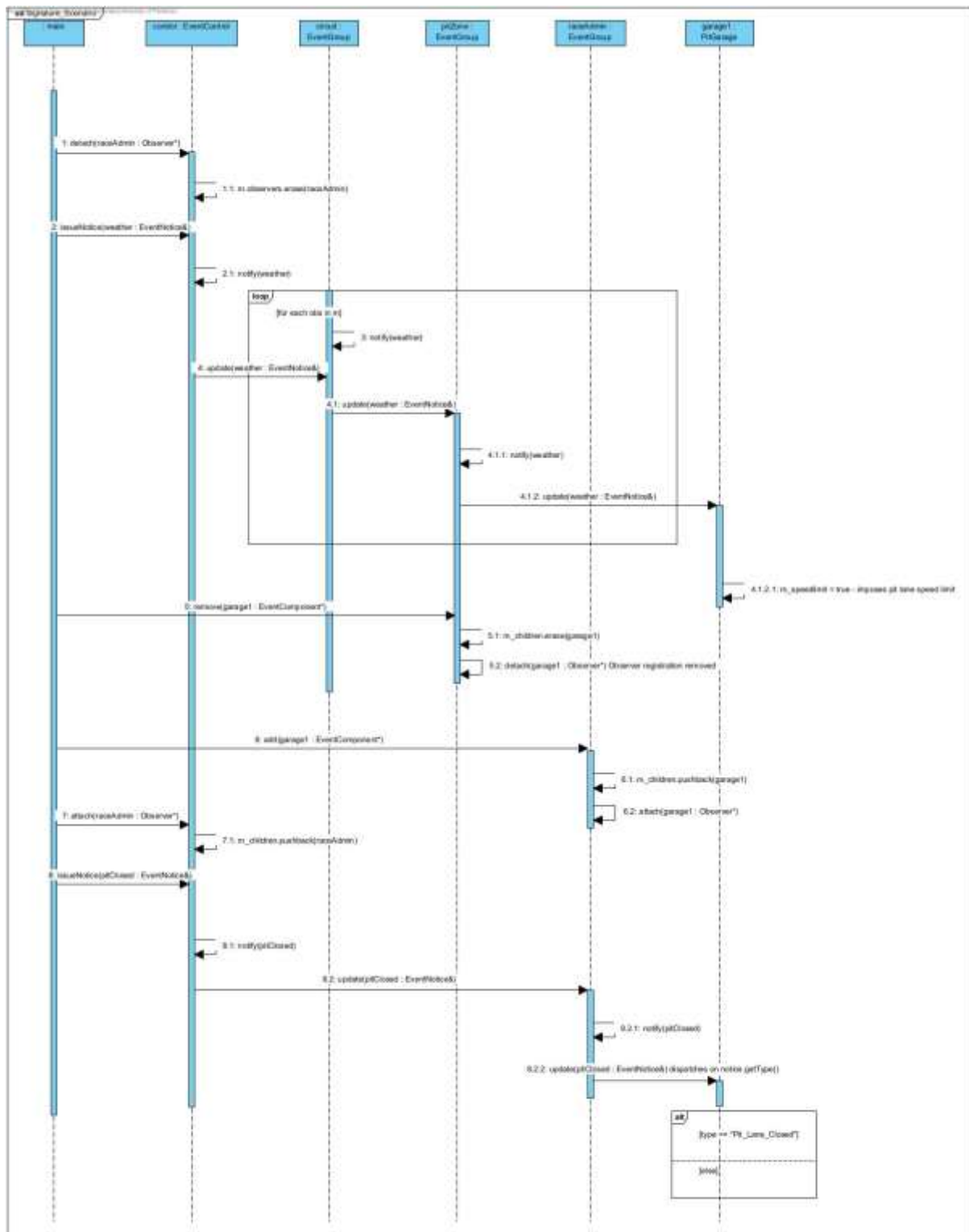
5.2)



5.3)



5.4)



TASK6: Doxygen Documentation



TASK7: GitHub Workflow Reflection

Rayhaan created the repository, invited the team, and implemented Task 3 (Observer system), Task 4 (event rules/dynamic behaviour), and the Doxygen documentation. Tafadzwa implemented Task 2 (Composite structure), an initial Main.cpp and Makefile, and the Composite object diagram. Oratile built the sequence diagram portfolio, which exposed integration gaps in the Composite implementation that Rayhaan and Oratile fixed together, jointly reworking Main.cpp for the final end to end simulation.

Each member worked on a separate feature branch, merging into main once their piece was working, which kept the three concurrent workstreams from clashing on shared files like EventGroup.h..